

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-платформы маркетплейса электронной техники
с системой управления заказами и аналитикой»

Выполнил: Нестерчук Антон Васильевич

Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Нестерчуку Антону Васильевичу,

обучающемуся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-платформы маркетплейса электронной техники с системой управления заказами и аналитикой»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	8	03.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	04.04.2026	
Задача 1.3. Формулировка цели и задач работы	7	06.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории и формирование требований к маркетплейсу	7	08.04.2026	
Задача 2.2. Описание функциональности платформы (UseCase, бизнес-логика ролей)	7	10.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ERD и схемы БД)	8	14.04.2026	
Задача 2.4. Разработка макетов страниц интерфейса пользователей платформы	5	18.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры клиентской части приложения	8	21.04.2026	
Задача 3.2. Разработка серверной части, реализация аутентификации и авторизации	8	27.04.2026	
Задача 3.3. Реализация CRUD-интерфейса для пользователей, товаров и заказов	9	03.05.2026	
Задача 3.4. Реализация системы корзины покупателя и бизнес-логики обработки заказов	9	09.05.2026	
Задача 3.5. Разработка модуля отчетности и работы с файлами	7	15.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Деплой приложения на хостинг	4	26.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«30» марта 2026 г.

(подпись) А. С. Кружалов

Дата выдачи задания

«30» марта 2026 г.

Дата сдачи выполненной работы

«26» мая 2026 г.

Задание принял к исполнению

«31» марта 2026 г.

(подпись) А. В. Нестерчук

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих платформ электронной коммерции . . .	6
1.2 Анализ программных инструментов разработки	7
1.3 Формулировка цели и задач работы	9
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	10
2.1 Анализ целевой аудитории и формирование требований	10
2.2 Описание функциональности платформы	11
2.3 Проектирование модели данных	14
2.4 Разработка макетов страниц	17
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	22
3.1 Разработка базовой структуры клиентской части приложения .	22
3.2 Разработка серверной части, реализация аутентификации и авторизации	23
3.3 Реализация CRUD-интерфейсов для пользователей, товаров и заказов	23
3.4 Реализация системы корзины покупателя и бизнес-логики об- работки заказов	25
3.5 Разработка модуля отчетности и работы с файлами	26

ВВЕДЕНИЕ

Актуальность темы. Современный рынок розничной торговли электроникой характеризуется высоким уровнем конкуренции и постепенным переходом от классических интернет-магазинов к агрегаторам. Потребители все чаще предпочитают приобретать смартфоны, компьютерные комплектующие и периферию на платформах, объединяющих предложения от различных поставщиков, что позволяет сравнивать цены и выбирать оптимальные условия. Однако глобальные маркетплейсы зачастую перегружены товарами из других категорий, обладают сложной системой комиссий для продавцов и не предоставляют специализированных инструментов фильтрации, необходимых при выборе электронной техники.

В связи с этим возникает потребность в создании нишевых, профильных торговых платформ. Разработка специализированного маркетплейса электронной техники, предоставляющего малым и средним ритейлерам изолированные рабочие кабинеты, а администрации — прозрачные инструменты управления товарными потоками и аналитики, является актуальной задачей в сфере веб-разработки.

Проблематика. Существующие готовые системы управления контентом требуют значительных трудозатрат для адаптации под многопользовательскую архитектуру, где каждый продавец должен иметь независимый доступ к своим товарам и заказам без риска вмешательства в данные других участников. Использование современных веб-фреймворков позволяет решить эту проблему за счет проектирования гибкой ролевой модели, микросервисной архитектуры и оптимизированных реляционных баз данных.

Объектом исследования является организация процесса дистанционной торговли цифровой техникой в рамках многопользовательских платформ.

Предмет исследования — архитектурные подходы, алгоритмы агрегации данных и программные технологии, применяемые для разработки серверной и клиентской частей веб-маркетплейса.

Цель курсового проекта заключается в проектировании и программной реализации веб-платформы маркетплейса электронной техники, обеспечивающей взаимодействие продавцов и покупателей, а также сбор статистических данных для администратора.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих платформ электронной коммерции

Разработка платформы электронной коммерции по модели маркетплейса для продажи цифровой техники является сложной инженерной задачей. Специфика предметной области требует обработки глубоко вложенных характеристик товаров (для реализации системы сложных фильтров), обеспечения строгой изоляции данных независимых продавцов и высокой скорости отклика пользовательского интерфейса.

Анализ современного рынка готового программного обеспечения выявляет три основных подхода к созданию подобных платформ. Рассмотрим их технические особенности:

1. **Сборки на базе WordPress (WooCommerce).** Open Source⁽¹⁾ решение. Главный недостаток — использование потенциально большого количества атрибутов у сущностей в базе данных. При работе с большим каталогом электроники и множественными фильтрами производительность системы падает.
2. **Специализированные CMS (CS-Cart Multi-Vendor).** Коробочное решение с готовой логикой для маркетплейсов. Недостаток — устаревающая монолитная архитектура с серверным рендерингом, что приводит к постоянным перезагрузкам страниц и снижению отзывчивости интерфейса.
3. **Корпоративные решения (1C-Bitrix Enterprise).** Мощная платформа с интеграцией складского учета. Отличается высокой стоимостью лицензии и закрытостью ядра, что сильно усложняет и удорожает внедрение современных одностраничных приложений (SPA) на базе API.

Для структурирования полученных данных был проведен сравнительный анализ по ключевым критериям разработки. Результаты представлены в таблице 1.1.

(1) Распространяется под свободной лицензией GNU GPL.

Таблица 1.1 – Сравнительный анализ платформ электронной коммерции

Название	Тип архитектуры	Гибкость модели БД	Готовность к SPA/API
WordPress + WooCommerce	Монолит	Низкая	Низкая
CS-Cart Multi-Vendor	Монолит	Средняя	Средняя
1C-Bitrix Enterprise	Монолит + REST API	Высокая	Требует написания прослойки API

Детальный анализ существующих программных решений показал, что подавляющее большинство из них опирается на монолитную архитектуру и устаревшие подходы к рендерингу, что делает их медленными при работе с большими каталогами электроники. Использование готовых CMS накладывает ограничения на структуру базы данных, усложняя реализацию быстрых фильтров.

В связи с этим подтверждается целесообразность разработки собственного программного продукта. Разделение системы на независимый серверный API и клиентское приложение позволит добиться максимальной гибкости ролевой модели, высокой скорости отклика интерфейса и оптимизации работы с реляционной базой данных.

1.2 Анализ программных инструментов разработки

Для реализации проекта выбран современный стек технологий, обеспечивающий разделение приложения на независимые клиентскую и серверную части. Выбор инструментов определен требованиями к масштабируемости, скорости отклика интерфейса и безопасности данных:

- **Frontend-библиотека (React.js):** На современном рынке клиентской разработки доминируют такие фреймворки, как Angular, Vue.js и библиотека React. Angular обладает излишне жесткой архитектурой, которая избыточна для данного проекта, а Vue.js имеет менее обширную систему готовых инструментов для управления состояниями. Выбор библиотеки

React обусловлен компонентным подходом, использованием виртуальной древовидной структуры объектов, что обеспечивает высокую скорость перерисовки интерфейса каталога при применении множества фильтров, а также наличием официальных рекомендаций по разработке интерфейсов [1].

- **Стилизация (Tailwind CSS):** Для стилизации компонентов традиционно применяются CSS-препроцессоры или готовые UI-библиотеки, например, Bootstrap. Стоит отметить, что препроцессоры усложняют поддержку кода при росте проекта, а UI-библиотеки сильно ограничивают уникальность дизайна. Фреймворк Tailwind CSS был выбран для ускорения верстки интерфейса и минимизации объема CSS-кода за счет использования утилитарных классов, подробное описание которых приводится в документации [2].
- **Backend-платформа (Python + FastAPI):** При выборе серверной технологии рассматривались такие популярные решения, как Node.js и Python-фреймворки (Django, Flask). Node.js требует сторонних решений для строгой валидации данных, Django является тяжеловесным монолитом, а Flask не имеет встроенной асинхронности. В свою очередь, FastAPI обеспечивает высокую производительность за счет асинхронности и автоматическую генерацию документации с помощью Swagger, что необходимо для обработки запросов от множества пользователей одновременно [3].
- **СУБД (PostgreSQL):** Для хранения данных рассматривались нереляционные (MongoDB) и реляционные (MySQL, PostgreSQL) СУБД. Нереляционные базы не подходят для платформ электронной коммерции, так как не обеспечивают надежность финансовых транзакций. В сравнении с MySQL, выбор PostgreSQL аргументирован лучшей поддержкой сложных связей и наличием типа данных JSONB, что позволит эффективно хранить динамические характеристики электронной техники при сохранении высокой надежности реляционной структуры [4].

Для реализации дополнительного функционала клиентской части используется библиотека `react-router-dom`, обеспечивающая маршрутизацию страниц без перезагрузки браузера.

В серверной части приложения для взаимодействия с СУБД PostgreSQL применяется ORM-библиотека SQLAlchemy. Она позволяет абстрагироваться от прямых SQL-запросов, предотвращает уязвимости к SQL-инъекциям и эффективно работает с объектами базы данных на языке Python. Для обеспечения безопасности доступа пользователей используется протокол JSON Web Token. Его выбор обусловлен отдельной архитектурой веб-приложения: токены не требуют хранения сессий на сервере и позволяют безопасно передавать данные о ролях пользователей между независимыми клиентом и сервером.

1.3 Формулировка цели и задач работы

На основе проведенного анализа инструментов и выявленных проблем существующих маркетплейсов была сформулирована цель курсового проекта.

Цель работы — проектирование и разработка веб-платформы маркетплейса электронной техники, обеспечивающей взаимодействие продавцов и покупателей с системой аналитической отчетности.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ предметной области и обосновать выбор программных инструментов разработки;
2. Сформировать функциональные требования к платформе и описать бизнес-логику основных ролей пользователей;
3. Спроектировать модель данных и макеты пользовательских интерфейсов;
4. Разработать клиентскую часть приложения (карточки товаров, корзина и личные кабинеты);
5. Реализовать серверную часть с модулями аутентификации, управления каталогом и обработки заказов;
6. Выполнить развертывание разработанной системы на хостинге и подготовить отчетную документацию.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории и формирование требований

Анализ целевой аудитории позволяет выделить два основных сегмента пользователей. Первый сегмент — это розничные покупатели, заинтересованные в приобретении цифровой техники. Их главная потребность заключается в удобном поиске по специфическим техническим характеристикам и возможности сравнения предложений от разных поставщиков. Второй сегмент — представители малого и среднего бизнеса, нуждающиеся в доступном канале продажи своей продукции с минимальными техническими барьерами для входа.

Исходя из потребностей целевой аудитории, в проектируемой системе выделяются три ключевые ролевые модели с изолированными правами доступа:

- **Покупатель** — авторизованный пользователь, имеющий доступ к витрине товаров. Нуждается в инструментах фильтрации, управления корзиной заказов и отслеживания статусов текущих покупок через личный кабинет.
- **Продавец** — коммерческий пользователь, управляющий своими карточками товаров. Основные потребности продавца: публикация товаров с загрузкой медиафайлов, управление остатками, обработка поступивших заказов и получение агрегированной статистики продаж.
- **Администратор** — сотрудник платформы, обладающий полными правами. Отвечает за модерацию контента, блокировку нарушителей и управление глобальным справочником категорий техники.

На основе выделенных ролей был сформирован перечень различных требований к разрабатываемому программному продукту.

Функциональные требования:

1. Регистрация, аутентификация и авторизация пользователей с разграничением прав доступа на основе ролей.
2. Реализация CRUD-интерфейсов для управления профилями, каталогом товаров и категориями.

3. Наличие логики управления корзиной покупок и оформления заказа.
4. Маршрутизация заказов: продавец должен иметь доступ только к тем заказам, которые содержат добавленные им товары.
5. Возможность загрузки и хранения медиафайлов.
6. Выполнение агрегации данных на стороне сервера для формирования сводной аналитической отчетности продавца (подсчет выручки, количества заказов).

Нефункциональные требования:

1. Производительность: клиентская часть должна функционировать как одностраничное приложение (SPA), обеспечивая фильтрацию каталога без перезагрузки страницы.
2. Безопасность: пароли пользователей должны храниться в базе данных в виде криптографических хешей; передача сессионных данных должна осуществляться через подписанные JWT-токены.
3. Масштабируемость: структура базы данных должна быть нормализована для предотвращения аномалий при добавлении новых продавцов или характеристик товара.
4. Эргономичность: понятный интерфейс, удобная навигация и логичность расположения элементов в приложении.

Таким образом, проведенный анализ целевой аудитории и составление требований формируют четкий вектор для последующего моделирования бизнес-процессов и проектирования физической структуры реляционной базы данных.

2.2 Описание функциональности платформы

Для формализации функциональных требований и проектирования бизнес-логики разрабатываемой веб-платформы применена методология пользовательских историй. Данный подход позволяет описать функциональность программного продукта с позиции конечного потребителя. Рассмотрим ключевые сценарии взаимодействия для каждой роли в системе.

Сценарии для базовой роли **«Пользователь системы»**:

- *Как новый пользователь*, я хочу зарегистрироваться в системе, чтобы получить доступ к функционалу покупателя или продавца.
- *Как зарегистрированный пользователь*, я хочу авторизоваться в системе по своим учетным данным, чтобы безопасно управлять своим профилем.

Сценарии для роли «**Покупатель**»:

- *Как покупатель*, я хочу просматривать витрину и использовать фильтры, чтобы быстро находить нужную мне электронику.
- *Как покупатель*, я хочу переходить в карточку товара и добавлять его в корзину, чтобы формировать список покупок.
- *Как покупатель*, я хочу оформлять заказ из корзины и отслеживать его статус в личном кабинете, чтобы контролировать процесс доставки.

Сценарии для роли «**Продавец**»:

- *Как продавец*, я хочу иметь доступ к единой панели управления, чтобы централизованно вести коммерческую деятельность.
- *Как продавец*, я хочу управлять товарами и загружать к ним медиафайлы, чтобы качественно оформлять витрину.
- *Как продавец*, я хочу изменять статусы заказов и просматривать статистику продаж с возможностью выгрузки отчета в формате CSV, чтобы анализировать выручку.

Сценарии для роли «**Администратор**»:

- *Как администратор*, я хочу управлять категориями техники, чтобы поддерживать структурированность каталога.
- *Как администратор*, я хочу модерировать учетные записи пользователей, чтобы пресекать нарушения правил платформы.

На основе сформированных функциональных требований и пользовательских сценариев была разработана UML-диаграмма вариантов использования

(рисунок 2.1). Данная модель визуализирует границы маркетплейса и детализирует архитектуру взаимодействия ролей с использованием механизмов наследования и расширенной нотации связей.

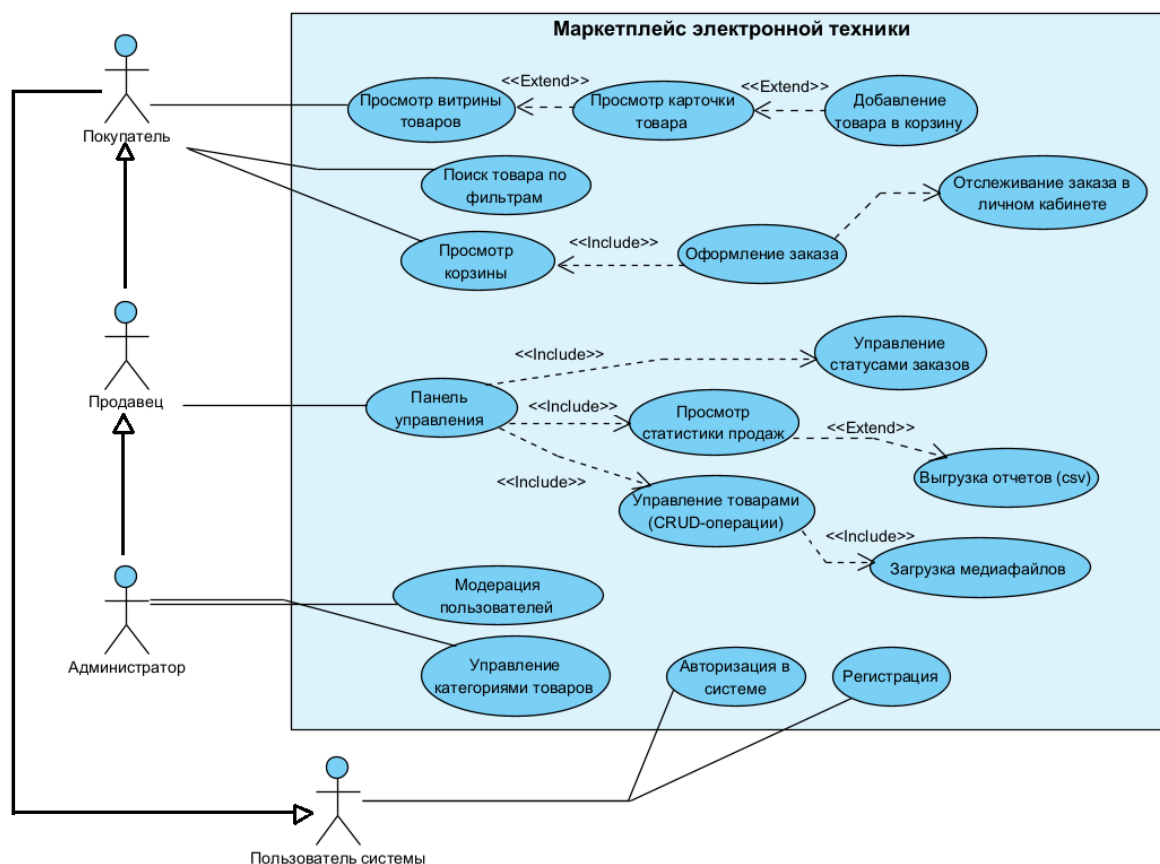


Рисунок 2.1 – Диаграмма вариантов использования (Use Case)

В системе реализована иерархия акторов: базовая абстракция «Пользователь системы» берет на себя общие прецеденты авторизации и регистрации, а остальные роли наследуют эти права. Бизнес-логика изобилует сложными зависимостями. Например, панель управления у продавцов включает в себя модули управления заказами, товарами и просмотра статистики. В свою очередь, создание карточки товара обязательно требует загрузки медиафайлов. Отношения расширения применены для дополнительного функционала, который вызывается опционально: выгрузка CSV-отчета из статистики или отслеживание заказа после его оформления. Разработанная поведенческая модель послужит надежным фундаментом для следующего этапа — проектирования физической структуры реляционной базы данных.

2.3 Проектирование модели данных

При проектировании инфологической модели были выделены следующие ключевые сущности:

- `roles` и `users` — таблицы для реализации системы авторизации и управления правами доступа.
- `categories` и `products` — таблицы для формирования структуры каталога и хранения карточек товаров. Каждая карточка товара жестко привязана к продавцу и категории.
- `orders` и `order_items` — таблицы для реализации бизнес-логики покупок.

Особое внимание при проектировании было уделено архитектурной проблеме заказов: в классическом интернет-магазине статус присваивается всему заказу целиком, однако в маркетплейсе один заказ может содержать товары от разных продавцов. Для решения этой проблемы статус заказа был вынесен на уровень позиций заказа. Это позволяет каждому продавцу изменять статус доставки только своего товара, не влияя на общую корзину покупателя.

Логическая структура базы данных со связями типа «один-ко-многим» представлена на ER-диаграмме (рисунок 2.2).

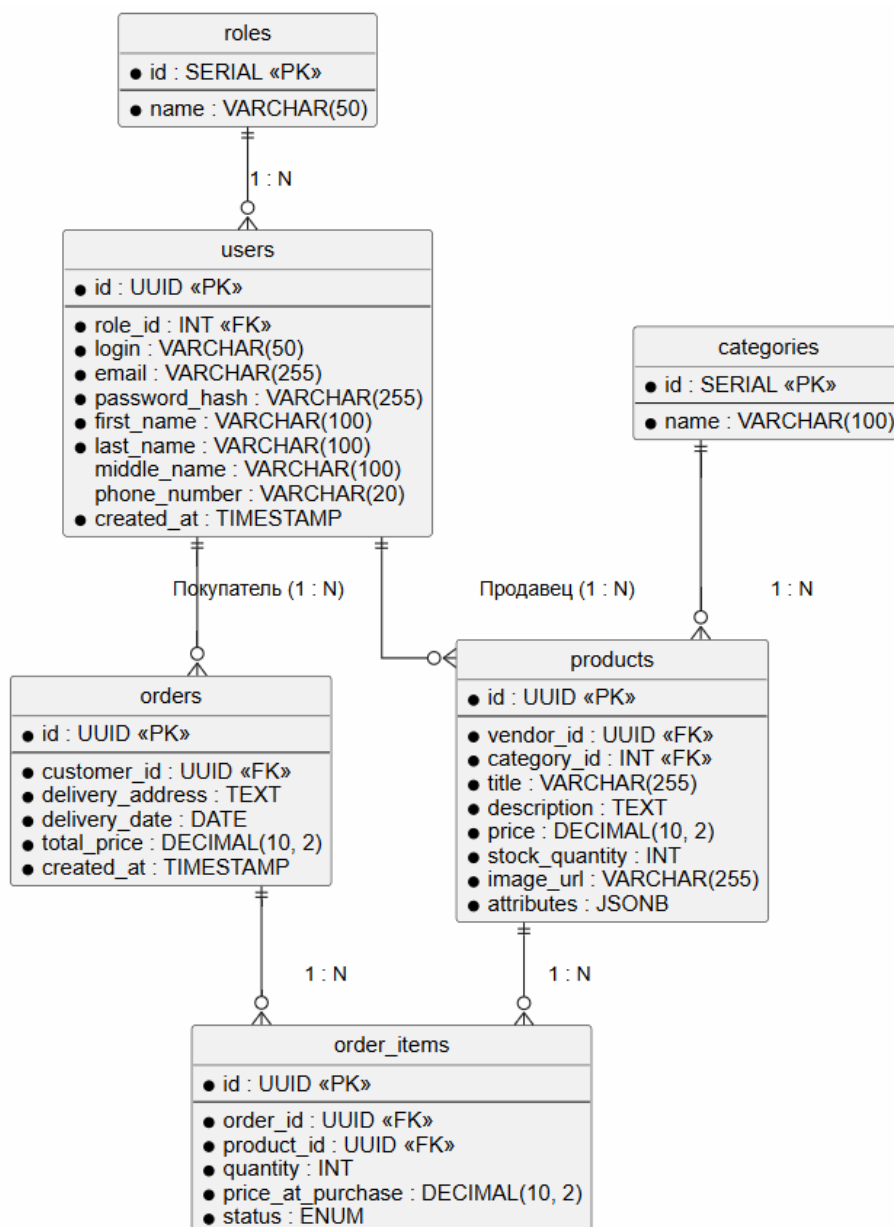


Рисунок 2.2 – ER-диаграмма базы данных

Для программной реализации спроектированная модель была нормализована до 3-й нормальной формы. Ниже (таблицы 2.1, 2.2, 2.3, 2.4) представлены физические схемы основных таблиц базы данных.

Таблица 2.1 – Структура таблицы пользователей (users)

Поле	Тип данных	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
role_id	INT	FOREIGN KEY	Ссылка на роль пользователя
login	VARCHAR(50)	UNIQUE, NOT NULL	Логин для входа

Продолжение таблицы 2.1

Поле	Тип данных	Ограничение	Описание
email	VARCHAR(255)	UNIQUE, NOT NULL	Электронная почта
password_hash	VARCHAR(255)	NOT NULL	Хеш пароля (bcrypt)
first_name	VARCHAR(100)	NOT NULL	Имя
last_name	VARCHAR(100)	NOT NULL	Фамилия
middle_name	VARCHAR(100)	NULL	Отчество (необязательно)
phone_number	VARCHAR(20)	NULL	Телефон для связи
created_at	TIMESTAMP	DEFAULT NOW()	Дата регистрации

Таблица 2.2 – Структура таблицы товаров (products)

Поле	Тип данных	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
vendor_id	UUID	FOREIGN KEY	Ссылка на продавца
category_id	INT	FOREIGN KEY	Ссылка на категорию
title	VARCHAR(255)	NOT NULL	Наименование товара
description	TEXT	NULL	Подробное описание
price	DECIMAL(10,2)	CHECK > 0	Стоимость товара
stock_quantity	INT	DEFAULT 0	Остаток на складе
image_url	VARCHAR(255)	NULL	Путь к файлу изображения
attributes	JSONB	NOT NULL	Динамические характеристики

Таблица 2.3 – Структура таблицы заказов (orders)

Поле	Тип данных	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
customer_id	UUID	FOREIGN KEY	Ссылка на покупателя
delivery_address	TEXT	NOT NULL	Полный адрес доставки
delivery_date	DATE	NOT NULL	Плановая дата доставки
total_price	DECIMAL(10,2)	NOT NULL	Итоговая стоимость заказа
created_at	TIMESTAMP	DEFAULT NOW()	Дата и время оформления

Таблица 2.4 – Структура таблицы позиций заказа (order_items)

Поле	Тип данных	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
order_id	UUID	FOREIGN KEY	Ссылка на общий чек (заказ)
product_id	UUID	FOREIGN KEY	Ссылка на купленный товар
quantity	INT	CHECK > 0	Количество единиц товара
price_at_purchase	DECIMAL(10,2)	NOT NULL	Цена на момент покупки
status	ENUM	NOT NULL	Статус отправки продавцом

Спроектированная реляционная модель исключает дублирование данных и полностью готова к интеграции с серверной частью приложения через ORM SQLAlchemy.

2.4 Разработка макетов страниц

Для визуализации пользовательского интерфейса и проектирования путей взаимодействия были разработаны низкодетализированные структурные макеты основных страниц (Wireframes). Данный подход позволяет сфокусироваться на эргономике, расположении элементов управления и информационной архитектуре до начала этапа программной верстки. Проектирование макетов осуществлялось с учетом компонентного подхода, который будет реализован в дальнейшем.

Главная страница является точкой входа для покупателей. В верхней части расположена глобальная панель навигации с модулем глобального поиска, кнопками авторизации и доступа к корзине. Ниже расположен рекламный блок с призывом к действию (СТА). Ключевым элементом интерфейса является панель фильтрации и блок категорий, позволяющие покупателю отсеивать технику по заданным критериям. Основное пространство занимает модульная сетка популярных товаров. Макет главной страницы представлен на рисунке 2.3.

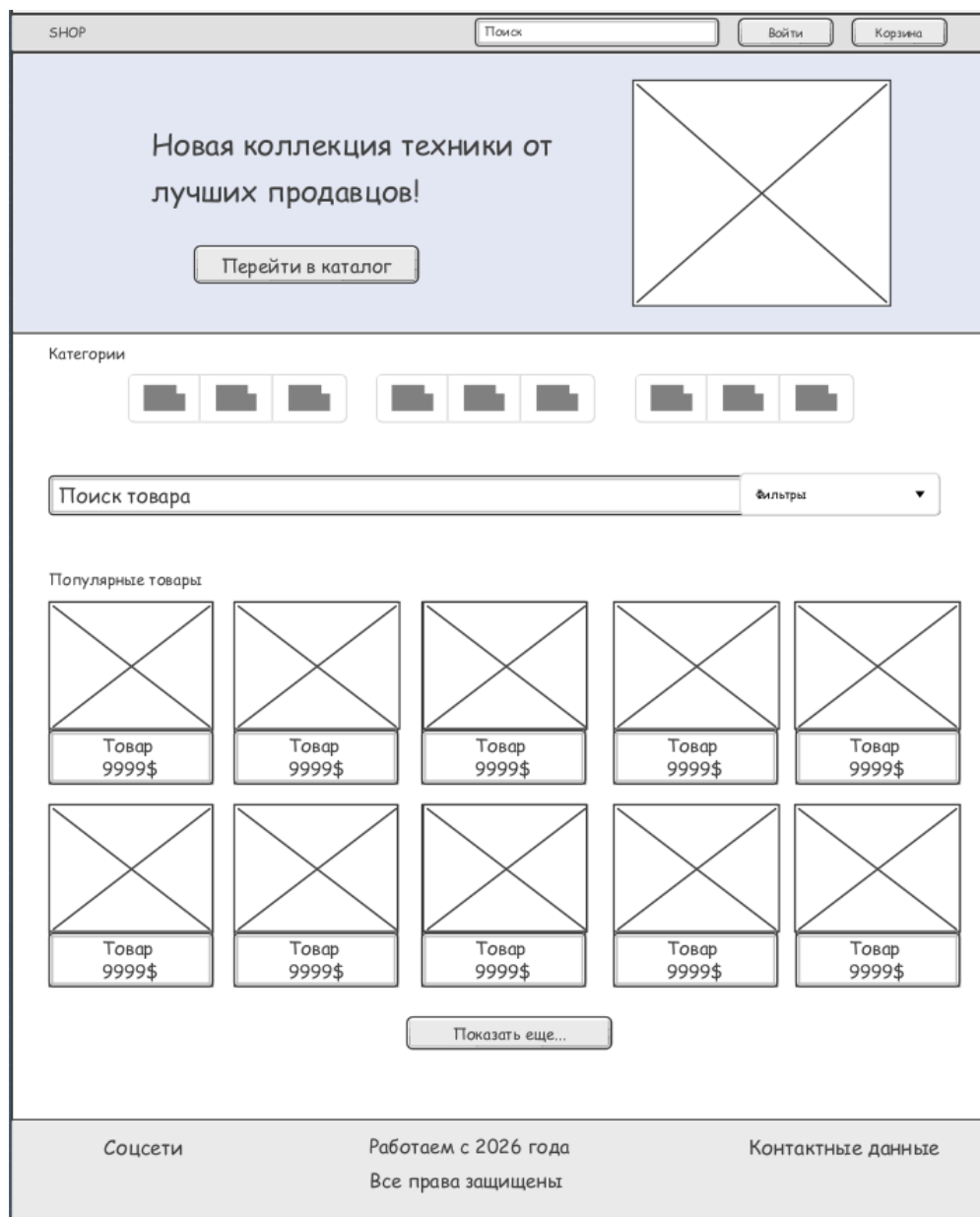


Рисунок 2.3 – Макет главной страницы

Карточка товара разработана с акцентом на детальное представление сложной электроники. Макет (рисунок 2.4) включает крупную область для просмотра фотографий товара. В информационной панели, помимо цены и названия, предусмотрен вывод имени продавца и динамических селекторов характеристик. Ниже располагаются блоки с текстовым описанием, техническими спецификациями и рекомендательной системой товаров из той же категории.



Рисунок 2.4 – Макет карточки товара

Интерфейс корзины (рисунок 2.5) спроектирован для удобной агрегации товаров от различных продавцов. Пользователь имеет возможность управлять количеством позиций и удалять неактуальные товары. Правая боковая панель зафиксирована и отображает информацию о заказе с кнопкой перехода к оформлению заказа.

SHOP

Корзина

	Название товара описание Удалить - 1 + 999\$
	Название товара описание Удалить - 2 + 999\$
	Название товара описание Удалить - 1 + 999\$

Итого

Товары (4шт) 2997\$
Доставка Бесплатно
К оплате 2997\$

Соцсети

Работаем с 2026 года
Все права защищены

Контактные данные

Рисунок 2.5 – Макет корзины

Для продавцов спроектирован изолированный интерфейс панели управления (рисунок 2.6). Макет имеет классическую для B2B-систем структуру, соответствующую современным паттернам проектирования коммерческих интерфейсов [5]: левое боковое меню для навигации между разделами и широкую рабочую область. В ней визуализированы графики агрегированной статистики продаж с функцией экспорта данных в формате CSV. Ниже расположены списки для выполнения CRUD-операций над карточками товаров и управления статусами поступивших заказов.

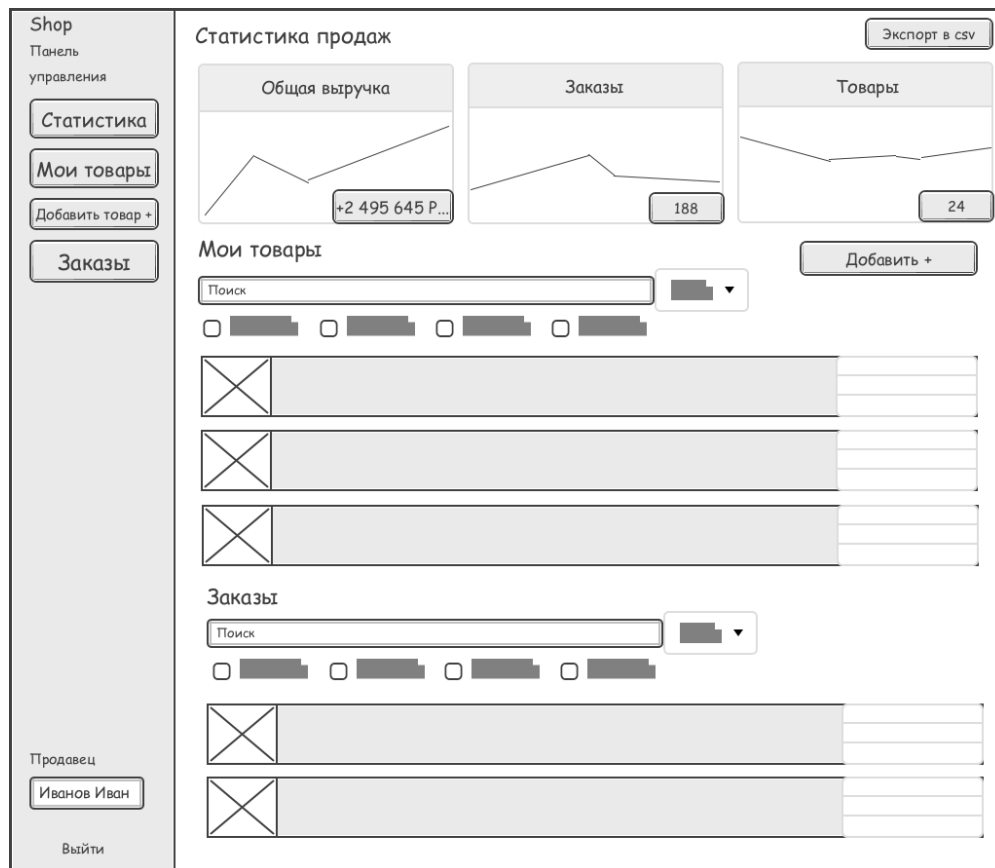


Рисунок 2.6 – Макет панели управления продавца

Таким образом, завершен этап архитектурного планирования, и подготовлена исчерпывающая база для программной реализации серверной и клиентской частей маркетплейса в следующей главе.

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры клиентской части приложения

Инициализация и сборка клиентской части веб-приложения была выполнена с помощью современного инструмента Vite в связке с библиотекой **React.js**. Данный выбор обеспечил высокую скорость перезагрузки модулей при разработке и оптимизацию итогового бандла. Для стилизации интерфейсов применен фреймворк Tailwind CSS, реализующий концепцию Utility-First, что позволило отказаться от громоздких каскадных таблиц стилей в пользу атомарных классов.

Архитектура клиентской части построена на основе компонентного подхода. Исходный код был логически разделен на следующие директории:

- `components` — переиспользуемые элементы пользовательского интерфейса (шапка, подвал, карточка товара, боковые панели дашбордов);
- `pages` — уникальные страницы-компоненты (главная витрина товаров, страница авторизации, корзина, личный кабинет);
- `api` — модуль инкапсуляции сетевых запросов и настройки связи с серверной частью.

Маршрутизация в одностраничном приложении (SPA) реализована с помощью библиотеки `react-router-dom`. Был спроектирован единый диспетчер маршрутов в главном файле `App.jsx`, обеспечивающий переходы по страницам без перезагрузки браузера.

Ключевым элементом базовой структуры является централизованный HTTP-клиент на базе библиотеки **Axios**. Для обеспечения безопасности и прозрачной работы с JWT-токенами был реализован механизм перехватчиков запросов. Данный архитектурный паттерн позволяет автоматически извлекать токен доступа из локального хранилища браузера (`localStorage`) и внедрять его в заголовки перед отправкой запроса на сервер. Использование централизованного HTTP-клиента исключило дублирование кода при обращении к эндпоинтам на сервере и заложило надежный фундамент для дальнейшей реализации модулей авторизации и управления контентом.

3.2 Разработка серверной части, реализация аутентификации и авторизации

В соответствии с задачей 3.2, серверная часть веб-приложения была разработана с использованием фреймворка **FastAPI**. Взаимодействие с реляционной системой управления базами данных PostgreSQL реализовано с помощью **SQLAlchemy ORM**, что позволило абстрагироваться от прямых SQL-запросов и обеспечить безопасность от инъекций.

Для организации надежной системы авторизации был выбран стандарт **OAuth2** с использованием JSON Web Tokens. Данный подход позволяет реализовать *Stateless*-архитектуру, где сервер не хранит активные сессии пользователей в оперативной памяти. Это существенно повышает масштабируемость платформы.

В БД была спроектирована модель пользователей (*users*) со связью к таблице ролей (*roles*). Пароли пользователей не сохраняются в открытом виде: перед записью в СУБД они проходят криптографическое хеширование с использованием алгоритма *bcrypt*.

При успешной проверке учетных данных на эндпоинте `/auth/login` сервер генерирует подписанный JWT-токен, в полезную нагрузку которого внедряется уникальный идентификатор пользователя.

Специфика разрабатываемого продукта требует строгого разделения прав доступа между покупателями, продавцами и администраторами. Для решения этой задачи в FastAPI был реализован параметризованный класс-зависимость *RoleChecker*. Такой механизм в проекте гарантирует, что валидация токена и проверка роли пользователя (например, `'RoleChecker([3])'`) будут выполнены до передачи управления бизнес-логике функции. В случае несовпадения прав доступа, система автоматически прервет обработку запроса и вернет клиенту статус-код 403 *Forbidden*, обеспечивая надежную защиту платформы.

3.3 Реализация CRUD-интерфейсов для пользователей, товаров и заказов

В приложении был реализован комплекс модулей, обеспечивающих CRUD-операции для ключевых сущностей в системе. Логика управления данными была сегментирована в зависимости от ролевой принадлежности

авторизованного пользователя.

Управление пользователями реализовано в рамках панели администратора. Был спроектирован интерфейс модерации, представляющий собой агрегированную таблицу с данными об атрибутах (логин, ФИО, контактные данные, текущая роль). Администратор наделен правами редактирования профилей для исправления ошибочных данных, изменения системных ролей, а также безвозвратного удаления учетных записей при нарушении правил платформы. Благодаря каскадному удалению на уровне базы данных, при удалении учетной записи автоматически очищаются связанные с ним товары и истории заказов.

Интерфейс управления каталогом товаров разделен между продавцами и администраторами. Для первой роли реализована изолированная B2B-панель. При создании и редактировании карточки товара продавец заполняет базовые поля и загружает графическое изображение. Загрузка медиафайлов реализована через отдельный независимый эндпоинт, возвращающий URL-адрес сохраненного на сервере изображения.

Особое внимание при разработке формы товара было уделено работе с динамическими характеристиками, хранящимися в СУБД в типе данных JSONB. На клиентской части был спроектирован визуальный конструктор параметров. При вводе значений через запятую, алгоритм фронтенд-части автоматически преобразует строку в массив, что в дальнейшем позволяет системе корректно отображать эти параметры в виде кликабельных селекторов на витрине маркетплейса.

Модуль управления заказами также адаптирован под специфику многопользовательской торговли:

- **Для покупателя:** реализован интерфейс истории покупок в личном кабинете. Покупатель видит общую сумму чека, плановую дату доставки и детализированный список купленных товаров с индивидуальными статусами их логистики.
- **Для продавца:** спроектирована панель обработки входящих заявок. Система автоматически фильтрует глобальную таблицу позиций заказов, отображая продавцу только те товары, которые принадлежат его магазину. Продавец имеет возможность изменять статус логистики через выпадающий список, что отражается в кабинете покупателя.

- **Для администратора:** реализован глобальный мониторинг всех транзакций платформы с информацией о покупателях и возможностью изменения статуса любых позиций (например, для разрешения спорных ситуаций).

Реализованный комплекс CRUD-операций полностью покрывает жизненный цикл сущностей приложения, обеспечивая надежное хранение данных и эргономичное взаимодействие пользователей с системой.

3.4 Реализация системы корзины покупателя и бизнес-логики обработки заказов

В рамках задачи был спроектирован и реализован модуль управления пользовательской корзиной и процесс оформления заказа. Для оптимизации нагрузки на серверную инфраструктуру и повышения отзывчивости интерфейса, промежуточное хранение данных корзины реализовано на стороне клиента с использованием `localStorage` браузера.

При нажатии на кнопку «Добавить в корзину» в карточке товара, клиентское приложение формирует объект, включающий идентификатор товара, количество и выбранные пользователем конфигурации (динамические селекторы цвета, памяти и т.д.). Для предотвращения дублирования позиций алгоритм проверяет корзину на наличие аналогичного товара с такими же выбранными параметрами. Сравнение объектов реализовано путем их сериализации в формат JSON. В случае совпадения увеличивается счетчик количества, в противном случае создается новая запись. Также на этапе добавления внедрена блокировка, не позволяющая превысить доступный складской остаток товара.

Интерфейс страницы корзины разделен на две логические области: список выбранных товаров с элементами управления количеством и панель итогового чека. При изменении количества товаров алгоритм динамически пересчитывает общую сумму и обновляет локальное хранилище.

Бизнес-логика оформления заказа инициируется при заполнении пользователем адреса и плановой даты доставки. При отправке формы клиентское приложение передает на сервер сформированный массив позиций корзины. Обработка заказа на стороне бэкенда представляет собой сложную транзакцию, выполняющуюся в несколько этапов:

1. **Проверка консистентности:** сервер повторно валидирует наличие за-

прошенных товаров в базе данных и проверяет, не превышает ли запрашиваемое количество текущие складские остатки.

2. **Списание остатков:** в случае успешной валидации система уменьшает значение поля `stock_quantity` у соответствующих товаров, исключая возможность их продажи другим пользователям.
3. **Формирование чека:** алгоритм рассчитывает итоговую стоимость корзины на стороне сервера и создает запись заказа (`Order`).
4. **Привязка позиций:** для фиксации связи «один-ко-многим», система использует метод `flush()` из ORM `SQLAlchemy`. Он принудительно отправляет SQL-запрос в базу данных без закрытия транзакции, генерируя уникальный `UUID` заказа. Этот идентификатор присваивается всем дочерним позициям (`OrderItem`), после чего транзакция окончательно фиксируется методом `commit()`.

При успешном завершении транзакции локальная корзина клиента очищается, а заказ становится доступен в кабинете покупателя и панелях управления соответствующих продавцов.

3.5 Разработка модуля отчетности и работы с файлами

Был спроектирован и реализован модуль коммерческой аналитики, интегрированный в панель управления продавца. Целью данного модуля является предоставление агрегированных финансовых показателей и генерация отчетной документации.

Для просмотра статистических данных на клиенте был разработан дашборд (рисунок 3.1), отображающий общую выручку, количество уникальных заказов и объем проданных единиц товара. Расчет метрик выполняется на стороне базы данных с применением агрегатных функций (подсчет уникальных чеков осуществляется через связку `func.count(func.distinct(...))`). Это переносит вычислительную нагрузку с приложения на СУБД, обеспечивая высокую скорость отклика.

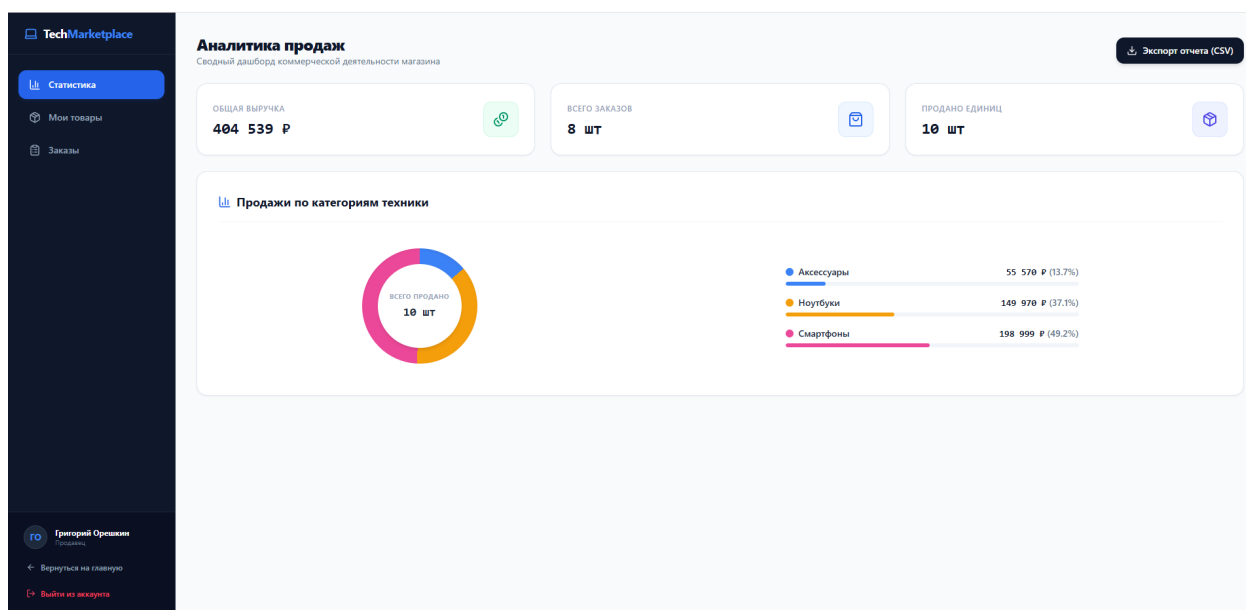


Рисунок 3.1 – Раздел панели управления продавца с аналитической составляющей

Визуализация структуры продаж реализована в виде динамической круговой диаграммы. На клиентской стороне алгоритм рассчитывает процентную долю каждой категории в общей выручке продавца и генерирует CSS-правило `conic-gradient`. Использованный подход позволил отказаться от использования сторонних библиотек для отрисовки графиков.

Особое внимание было уделено функционалу выгрузки финансовой отчетности в формате CSV. Пример экспортируемой таблицы на рисунке 3.2. Бизнес-логика экспорта реализована с учетом принципов оптимизации ввода-вывода и экономии дискового пространства сервера:

1. **Сбор данных:** сервер выполняет SQL-запрос с объединениями (*JOIN*) таблиц заказов, позиций и товаров, отфильтрованный по идентификатору текущего продавца и отсортированный по убыванию даты транзакции.
2. **Виртуализация файла:** вместо физического сохранения CSV-документа на жестком диске сервера, используется модуль `io.StringIO`, создающий текстовый буфер непосредственно в оперативной памяти.
3. **Кодирование:** для корректного отображения кириллицы в начало потока принудительно записывается маркер последовательности байтов `\ufeff`. Запись строк данных осуществляется с применением разделителя «точка с запятой».

4. **Подсчет итогов и стриминг:** в процессе итерации по списку продаж алгоритм рассчитывает итоговую выручку и дописывает ее в конец файла. Готовый буфер кодируется в байты и отправляется клиенту через класс `StreamingResponse`. Поточковая передача данных защищает сервер от переполнения оперативной памяти в случае генерации отчетов с сотнями тысяч строк.

	A	B	C	D	E	F	G
1	Дата заказа	Наименование товара	Количество (шт)	Цена за единицу (руб)	Сумма (руб)	Статус доставки	
2	04.06.2026 14:40	Смартфон Apple iPhone 15 Pro 512 ГБ, Dual nano	1	83000.0	83000.0	В обработке	
3	03.06.2026 14:03	Оригинал Сетевое зарядное устройство Samsu	1	1900.0	1900.0	В обработке	
4	02.06.2026 19:41	Беспроводные наушники Apple AirPods Pro 3	2	17890.0	35780.0	В обработке	
5	02.06.2026 19:40	Ноутбук HONOR MB X14 2025 i5-12450H 16+512	3	49990.0	149970.0	В пути	
6	02.06.2026 19:34	Смартфон Samsung Galaxy A57 5G 8/128 ГБ, nano	1	32999.0	32999.0	Доставлен	
7	02.06.2026 19:33	Смартфон Apple iPhone 15 Pro 512 ГБ, Dual nano	1	83000.0	83000.0	В обработке	
8	02.06.2026 19:30	Смартфон Samsung Galaxy A57 5G 8/128 ГБ, nano	2	32999.0	65998.0	Отменен	
9	02.06.2026 19:28	Беспроводные наушники Apple AirPods Pro 3	1	17890.0	17890.0	В пути	
10							
11	ИТОГОВЫЕ ПОКАЗАТЕЛИ ЗА ВСЕ ВРЕМЯ:						
12	Всего уникальных заказов:		8				
13	Заказано товаров:		10				
14	Общая выручка:	404539.0 руб.					
15							
16							

Рисунок 3.2 – Пример экспортируемой таблицы в формате csv

Принятые архитектурные решения обеспечили высокую производительность модуля отчетности, гарантируя продавцам мгновенный доступ к актуальной аналитике и безопасный экспорт коммерческих данных без создания избыточной нагрузки на сервер.

Исходный код разработанного веб-приложения размещен в системе контроля версий GitHub [6].

СПИСОК ЛИТЕРАТУРЫ

- [1] React Documentation: Quick Start [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://react.dev> (дата обращения: 29.04.2026).
- [2] Tailwind CSS Documentation: Utility-First Fundamentals [Electronic resource]. — [S. l. : s. n.]. — URL: <https://tailwindcss.com/docs> (online; accessed: 29.04.2026).
- [3] FastAPI Documentation: Professional API development [Electronic resource]. — [S. l. : s. n.]. — URL: <https://fastapi.tiangolo.com> (online; accessed: 29.04.2026).
- [4] PostgreSQL Documentation: JSON Types [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.postgresql.org/docs/current/datatype-json.html> (online; accessed: 29.04.2026).
- [5] Проектирование B2B-интерфейсов: паттерны, дашборды и аналитика [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://uxpub.ru/b2b-dashboard-design> (дата обращения: 04.05.2026).
- [6] Репозиторий GitHub с кодом проекта [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://github.com/tpAnswer4yourself/web-course-project/tree/main> (дата обращения: 04.06.2026).